부록 · 과정 도구 명령 모음

Docker Compose 공통 — 띄우기 · 보기 · 멈추기 · 지우기

4~8주차의 모든 도구가 docker-compose.yml 로 뜹니다. 어느 도구든 이 명령들로 다룹니다. compose 파일이 있는 폴더에서 칩니다.

자주 쓰는 명령

명령	하는 일	확인한 결과
docker compose up -d	정의된 서비스를 모두 띄움(뒤에서)	컨테이너마다 Started
docker compose ps	지금 떠 있는 서비스 상태	Up · (healthy) · (health: starting)
docker compose ps -a	멈춘 것까지	Exited (0) … 도 보임
docker compose logs --tail 30 서비스	그 서비스 로그 끝 30줄	준비 완료 표시를 여기서 찾는다
docker compose logs -f 서비스	로그를 실시간으로 따라감	Ctrl+C 로 빠져나옴
docker compose exec 서비스 명령	떠 있는 컨테이너 안에서 명령 실행	아래 도구별 명령 대부분이 이 형태
docker compose up -d --scale 서비스=3	같은 서비스를 여러 개로	Spark 워커 2 → 3 (실측)
docker compose stop / start	멈춤 / 다시 켬 — 데이터 남음	Airflow: Exited → Up (실측)
docker compose down	컨테이너·네트워크 지움 — 볼륨은 남음	Network … Removed
docker compose down -v	볼륨까지 지움 — 데이터가 사라짐	Volume … Removed

짚을 점

- stop 과 down 은 다릅니다. stop 은 멈추기만 해서 start 로 그대로 돌아오고, down 은 컨테이너를 지웁니다. down -v 는 데이터(볼륨)까지 지우니, 실습 데이터를 남기고 싶으면 쓰지 않습니다.
- 여러 주차의 도구를 동시에 띄우면 포트가 겹칩니다. Spark 마스터 UI·Trino·Airflow 웹이 모두 8080 입니다. 이번 주에 안 쓰는 것은 docker compose stop 으로 멈춥니다(부록 「실습 트러블슈팅」의 포트 충돌).
- exec 결과를 파이프(|)나 파일로 넘길 때는 docker compose exec -T 처럼 -T 를 붙입니다. 터미널을 붙이지 않아 줄 끝 문자가 섞이지 않습니다.

부록 · 과정 도구 명령 모음

MinIO · mc — 버킷과 파일 다루기

4주차 레이크하우스의 저장소입니다. 버킷과 파일은 mc(MinIO 클라이언트) 컨테이너 안에서 다룹니다. 4-1 의 compose 로 띄운 상태를 기준으로 합니다.

먼저 — 이미지 이름이 바뀌었습니다 (2026-10-02 실측)

minio/minio · minio/mc 는 지금 내려받을 수 없습니다. Docker Hub 는 「repository does not exist」(404), quay.io 는 401 을 돌려줍니다. 같은 MinIO 를 빌드해 배포하는 pgsty/minio · pgsty/mc 로 이름만 바꾸면 그대로 동작합니다(실측: 콘솔 9001 · 상태 확인 200, 아래 명령 전부 정상). 과정 노트(4~7주차)와 슬라이드는 2026-10-03 에 이 이름으로 바꿨습니다.

```yaml
# 4-1 docker-compose.yml
    image: pgsty/minio
    image: pgsty/mc
```

연결과 버킷

```bash
docker compose exec mc mc alias set local http://minio:9000 minioadmin minioadmin
docker compose exec mc mc mb local/raw
docker compose exec mc mc ls local
```

```text
Added `local` successfully.
Bucket created successfully `local/raw`.
[2026-10-02 15:46:26 UTC]     0B raw/
```

자주 쓰는 명령 (앞에 docker compose exec mc 를 붙입니다)

명령	하는 일	확인한 결과
mc cp /data/hello.txt local/raw/hello.txt	파일 하나 올리기	`/data/hello.txt` -> `local/raw/hello.txt`
mc cp --recursive /data/orders/ local/raw/orders/	폴더째 올리기	d1.csv · d2.csv 두 개
mc ls --recursive local/raw	버킷 안 전부 보기	크기·날짜와 함께 3개
mc cat local/raw/hello.txt	파일 내용 보기	hello minio
mc stat local/raw/hello.txt	파일 정보	Size 12 B · Content-Type: text/plain
mc du local/raw	버킷 용량	38B 3 objects raw
mc find local/raw --name "*.csv"	이름으로 찾기	orders/d1.csv · orders/d2.csv
mc mirror /data/orders local/backup/orders	폴더를 맞춰 올리기(바뀐 것만)	두 번째 실행은 0 B 전송
mc rm local/raw/hello.txt	파일 지우기	Removed `local/raw/hello.txt`.
mc rm --recursive --force local/raw/orders/	폴더째 지우기	두 개 Removed
mc rb --force local/backup	버킷 지우기(안에 든 것까지)	Removed `local/backup` successfully.
mc admin info local	서버 상태	Drives: 1/1 OK · Network: 1/1 OK

막힐 때 (실측)

- mc mirror … local/backup/… 에서 「The specified bucket does not exist」— mirror 는 버킷을 만들어 주지 않습니다. mc mb local/backup 을 먼저 합니다.
- mc rb local/backup 에서 「is not empty. Retry this command with --force」— 안에 파일이 있으면 --force 가 있어야 지워집니다. 지우기 전에 mc ls 로 확인합니다.
- mc 이미지 안에는 grep·awk·sed·wget·curl 이 없습니다(head·cat 은 있음 — 실측). docker compose exec mc sh -c "mc ls … | grep …" 은 「grep: command not found」로 실패합니다. 거르기는 컨테이너 밖에서 합니다: docker compose exec -T mc mc ls --recursive local/warehouse | grep -c .parquet
- docker run 으로 mc 를 한 번만 쓸 때 docker run … pgsty/mc sh -c "…" 는 「`sh` is not a recognized command」로 실패합니다. 이미지가 mc 로 시작하기 때문입니다. --entrypoint sh 를 주고 -c 로 넘깁니다: docker run --rm --network 네트워크 --entrypoint sh pgsty/mc -c "mc alias set … && mc ls …"
- /data 는 mc 컨테이너 안의 경로입니다. 4-1 compose 가 내 컴퓨터의 ./data 폴더를 거기에 붙여 두었으므로, 올릴 파일은 내 컴퓨터의 ./data 에 넣습니다.

부록 · 과정 도구 명령 모음

Kafka — 토픽 · 메시지 · 컨슈머 그룹

6주차의 메시지 브로커입니다. 6-2 의 compose(apache/kafka:3.7.0, 컨테이너 이름 kafka)로 띄운 상태를 기준으로 합니다. 명령은 모두 컨테이너 안의 /opt/kafka/bin 에 있습니다.

준비 완료 확인

```bash
docker logs kafka 2>&1 | grep "Kafka Server started"
```

```text
[2026-10-02 15:47:34,385] INFO [KafkaRaftServer nodeId=1] Kafka Server started (kafka.server.KafkaRaftServer)
```

토픽

```bash
docker exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic orders --partitions 3 --replication-factor 1

docker exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
docker exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic orders
```

```text
Created topic orders.
orders
Topic: orders   TopicId: ljPpcjjvTkm2YHDHqRM7hA PartitionCount: 3       ReplicationFactor: 1
Configs:
        Topic: orders   Partition: 0    Leader: 1       Replicas: 1     Isr: 1
        Topic: orders   Partition: 1    Leader: 1       Replicas: 1     Isr: 1
        Topic: orders   Partition: 2    Leader: 1       Replicas: 1     Isr: 1
```

명령	하는 일	확인한 결과
kafka-topics.sh … --alter --topic orders --partitions 4	파티션 늘리기	출력 없이 성공
kafka-topics.sh … --alter --topic orders --partitions 2	파티션 줄이기 — 안 됨	「2 would not be an increase」 오류
kafka-configs.sh … --alter --entity-type topics --entity-name orders --add-config retention.ms=3600000	보존 기간 1시간	Completed updating config for topic orders.
kafka-configs.sh … --describe --entity-type topics --entity-name orders	바꾼 설정 보기	retention.ms=3600000
kafka-topics.sh … --delete --topic orders	토픽 지우기	목록에서 사라짐

메시지 보내고 받기 (키 포함)

```bash
printf 'u1:{"id":1}\nu2:{"id":2}\nu1:{"id":3}\n' | docker exec -i kafka /opt/kafka/bin/kaf
ka-console-producer.sh \
  --bootstrap-server localhost:9092 --topic orders --property parse.key=true --property key.se
parator=:

docker exec kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
  --topic orders --from-beginning --max-messages 3 --property print.key=true --property print.
partition=true
```

```text
Partition:0     u1      {"id":1}
Partition:0     u1      {"id":3}
Partition:2     u2      {"id":2}
Processed a total of 3 messages
```

같은 키(u1)는 같은 파티션(0)으로 갑니다. 그래서 키가 같은 메시지끼리는 순서가 지켜집니다.

컨슈머 그룹과 오프셋

```bash
docker exec kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:90
92 \
  --topic orders --group g1 --from-beginning --max-messages 3

docker exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --
list
docker exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --
describe --group g1
```

```text
console-consumer-75738
g1

Consumer group 'g1' has no active members.

GROUP  TOPIC   PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
g1     orders  0          2               2               0
g1     orders  1          0               0               0
g1     orders  2          1               1               0
g1     orders  3          0               0               0
```

LAG 가 0 이면 다 읽은 것입니다. 처음부터 다시 읽게 하려면 오프셋을 되돌립니다. --dry-run 으로 먼저 보고, --execute 로 실제로 바꿉니다. 되돌린 뒤 LAG 가 2·1 로 다시 생깁니다(실측).

```bash
docker exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:909
2 \
  --group g1 --topic orders --reset-offsets --to-earliest --execute
```

토픽에 몇 개 쌓였나 — 오프셋 보기

```bash
docker exec kafka /opt/kafka/bin/kafka-get-offsets.sh --bootstrap-server localhost:9092 --
topic orders-test
```

```
orders-test:0:2
```
text

「토픽:파티션:마지막 오프셋」입니다. 메시지 2개를 보낸 뒤라 2 입니다.

막힐 때 (실측)

- kafka-run-class.sh kafka.tools.GetOffsetShell → 「Could not find or load main class kafka.tools.GetOffsetShell」. Kafka 3.7 에서 옮겨졌습니다. 위의 kafka-get-offsets.sh 를 쓰거나, 클래스 이름을 org.apache.kafka.tools.GetOffsetShell 로 바꿉니다(둘 다 orders-test:0:2 로 같은 결과).
- 같은 이름으로 다시 만들면 「Topic 'orders' already exists」— 이미 있는 것입니다. --list 로 확인합니다.
- 파티션은 늘릴 수만 있고 줄일 수 없습니다. 줄이려면 토픽을 지우고 다시 만듭니다(메시지도 사라짐).
- --group 없이 읽으면 console-consumer-숫자 같은 그룹이 자동으로 생깁니다. 그룹 목록에 모르는 이름이 보이면 이것입니다.
- 기동 직후 Connection refused 는 브로커가 아직 준비 중인 것입니다. 「Kafka Server started」 줄을 보고 칩니다.

부록 · 과정 도구 명령 모음

Spark — 클러스터 확인 · 작업 제출

5주차의 분산 처리 엔진입니다. 5-2 의 compose(apache/spark:3.5.3, 마스터 spark-master + 워커 2개)로 띄운 상태를 기준으로 합니다. 마스터 웹 화면은 http://localhost:8080 입니다.

워커가 붙었나

```bash
curl -s http://localhost:8080/json/ | python3 -c "import sys,json;d=json.load(sys.stdin);p
rint(d['status'], d['aliveworkers'], d['cores'])"
```

```text
ALIVE 2 4
```

마스터 상태 ALIVE, 살아 있는 워커 2개, 코어 합계 4개(워커당 --cores 2)입니다. 웹 화면 「Workers」 칸과 같은 값입니다.

예제 작업 제출 — SparkPi

```bash
docker exec spark-master /opt/spark/bin/spark-submit \
  --master spark://spark-master:7077 \
  --class org.apache.spark.examples.SparkPi \
  /opt/spark/examples/jars/spark-examples_2.12-3.5.3.jar 10 2>/dev/null | tail -1
```

```text
Pi is roughly 3.141279141279141
```

내 파이썬 작업 제출

```python
# job.py — 클러스터에서 도는 가장 작은 PySpark 작업
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("hello-job").getOrCreate()
df = spark.range(1, 1_000_001)                    # 1 ~ 1,000,000
print("합계:", df.selectExpr("sum(id)").first()[0])
print("파티션 수:", df.rdd.getNumPartitions())
spark.stop()
```

```bash
docker cp job.py spark-master:/tmp/job.py
docker exec spark-master /opt/spark/bin/spark-submit --master spark://spark-master:7077 /tmp/j
ob.py 2>/dev/null
```

```text
합계: 500000500000
파티션 수: 2
```

자주 쓰는 명령

명령	하는 일	확인한 결과
docker exec spark-master /opt/spark/bin/spark-sql --master spark://spark-master:7077 -e "SELECT 1 + 1 AS two"	SQL 한 줄 바로 실행	2
docker compose up -d --scale spark-worker=3	워커 늘리기	aliveworkers 3

명령	하는 일	확인한 결과
docker compose logs --tail 2 spark-worker	워커가 마스터에 붙었는지	Successfully registered with master
docker logs spark-master | grep "Registering worker"	마스터 쪽에서 본 워커	with 2 cores, 2.0 GiB RAM

짚을 점

- spark-submit 의 2>/dev/null 은 진행 로그(INFO)를 숨겨 결과만 보려는 것입니다. 작업이 실패할 때는 이걸 빼고 다시 실행해 오류를 봅니다.
- 파이썬 파일은 컨테이너 안에 있어야 합니다. docker cp 로 넣거나, compose 에 폴더를 붙여(volumes) 둡니다.
- --master 를 빼면 클러스터가 아니라 그 컨테이너 혼자(local) 돌고, 웹 화면 「Running Applications」에 나타나지 않습니다.

부록 · 과정 도구 명령 모음

Trino — CLI 로 조회하기

4주차의 SQL 엔진입니다. 4-4 에서는 Iceberg 카탈로그를 붙여 쓰지만, 명령 사용법은 같습니다. 여기서는 Trino 에 기본으로 들어 있는 예제 카탈로그(tpch)로 확인했습니다. 컨테이너 이름은 과정과 같이 trino 입니다.

준비 완료 확인

```bash
docker logs trino 2>&1 | grep "SERVER STARTED"
docker exec trino trino --version
```

```text
… io.trino.server.Server        ======== SERVER STARTED ========
Trino CLI 483
```

자주 쓰는 명령 (docker exec trino trino …)

명령	하는 일	확인한 결과
--execute "SHOW CATALOGS"	붙어 있는 카탈로그	jmx · memory · system · tpcds · tpch (4-4 에서는 iceberg 가 더 보임)
--execute "SHOW SCHEMAS FROM tpch"	카탈로그 안 스키마	information_schema · sf1 · tiny …
--execute "SHOW TABLES FROM tpch.tiny"	스키마 안 표	customer · lineitem · nation · orders …
--execute "DESCRIBE tpch.tiny.region"	표 구조	regionkey bigint · name varchar(25) · comment
--execute "SELECT count(*) FROM tpch.tiny.nation"	조회	"25"
--catalog tpch --schema tiny --execute "SELECT name FROM nation ORDER BY name LIMIT 3"	카탈로그·스키마를 정해 두고 짧은 이름으로	ALGERIA · ARGENTINA · BRAZIL
--output-format CSV_HEADER --execute "…"	머리줄 있는 CSV 로	"nationkey","name" 다음 줄부터 값
--execute "SELECT * FROM system.runtime.nodes"	살아 있는 노드	버전 483 · active

보기 좋은 표로

```bash
docker exec trino trino --output-format ALIGNED --execute "SELECT regionkey, name FROM tpch.tiny.region"
```

```text
 regionkey |    name
-----------+-------------
         0 | AFRICA
         1 | AMERICA
         2 | ASIA
         3 | EUROPE
         4 | MIDDLE EAST
(5 rows)
```

짚을 점

- --execute 로 한 줄씩 실행하면 기본 출력이 따옴표 붙은 CSV 입니다. 사람이 읽을 때는 --output-format ALIGNED, 파일로 저장할 때는 CSV_HEADER 가 편합니다.
- 없는 표를 부르면 「Table 'tpch.tiny.nationx' does not exist」와 함께 줄·칸 위치를 짚어 줍니다(실측). 4-4 에서는 iceberg.스키마.표 처럼 카탈로그부터 적습니다.
- 대화형으로 쓰려면 docker exec -it trino trino 로 들어가 trino> 프롬프트에서 SQL 을 칩니다. 빠져나올 때는 quit.
- 과정 compose 는 trinodb/trino 를 태그 없이 씁니다. 그래서 받는 날에 따라 버전이 달라질 수 있습니다(2026-10-03 기준 483).

부록 · 과정 도구 명령 모음

Qdrant — REST 로 컬렉션·포인트 다루기

7주차의 벡터 DB 입니다. 7-3 의 docker run(컨테이너 이름 qdrant, 포트 6333)으로 띄운 상태를 기준으로 합니다. 모든 기능을 HTTP 로 부를 수 있어 curl 로 확인합니다. 대시보드는 http://localhost:6333/dashboard 입니다.

살아 있나

```bash
curl http://localhost:6333/
curl http://localhost:6333/readyz
curl http://localhost:6333/collections
```

```text
{"title":"qdrant - vector search engine","version":"1.19.1",…}
all shards are ready
{"result":{"collections":[]},"status":"ok",…}
```

컬렉션 만들고 넣고 찾기

```bash
curl -X PUT http://localhost:6333/collections/docs -H 'Content-Type: application/json' \
  -d '{"vectors": {"size": 4, "distance": "Cosine"}}'

curl -X PUT "http://localhost:6333/collections/docs/points?wait=true" -H 'Content-Type: applic
ation/json' -d '{"points": [
  {"id": 1, "vector": [0.9,0.1,0,0], "payload": {"title": "MySQL"}},
  {"id": 2, "vector": [0.1,0.9,0,0], "payload": {"title": "MongoDB"}},
  {"id": 3, "vector": [0.8,0.2,0.1,0], "payload": {"title": "PostgreSQL"}}]}'

curl -X POST http://localhost:6333/collections/docs/points/query -H 'Content-Type: applicatio
n/json' \
  -d '{"query": [1,0,0,0], "limit": 2, "with_payload": true}'
```

```text
{"result":true,"status":"ok",…}
{"result":{"operation_id":1,"status":"completed"},"status":"ok",…}
{"result":{"points":[{"id":1,"version":1,"score":0.9938837,"payload":{"title":"MySQL"}},{"id":
3,"version":1,"score":0.9630868,"payload":{"title":"PostgreSQL"}}]},"status":"ok",…}
```

[1,0,0,0] 과 방향이 가장 비슷한 MySQL(0.99)·PostgreSQL(0.96)이 나왔습니다. 점수는 코사인 유사도라 1 에 가까울수록 비슷합니다.

자주 쓰는 요청

명령	하는 일	확인한 결과
GET /collections/docs	컬렉션 정보	points_count 3 · size 4 · Cosine · status green
POST /collections/docs/points/count {"exact": true}	포인트 수	{"count":3}
POST /collections/docs/points/scroll {"limit": 2, "with_payload": true}	앞에서부터 훑기	2개 + next_page_offset 3

명령	하는 일	확인한 결과
DELETE /collections/docs	컬렉션 지우기	{"result":true}
GET /dashboard	웹 화면	HTTP 200

막힐 때 (실측)

- 「Vector dimension error: expected dim: 4, got 3」 — 컬렉션을 만들 때 정한 차원(size)과 넣는 벡터의 길이가 다릅니다. 7주차에서는 임베딩 모델이 내는 차원 수로 컬렉션을 만들어야 합니다.
- ?wait=true 를 빼면 넣자마자 응답이 와서, 바로 이어 찾을 때 아직 안 보일 수 있습니다. 실습에서는 붙여 둡니다.
- Connection refused(6333) 는 컨테이너가 꺼져 있거나 포트가 빠진 것입니다. 부록 「실습 트러블슈팅」 컨테이너·포트 편을 봅니다.

부록 · 과정 도구 명령 모음

Airflow — DAG 확인 · 시험 · 실행

8주차의 작업 예약 도구입니다. 8-1 처럼 공식 docker-compose.yaml(2.9.3)을 내려받아 airflow-init → up -d 로 띄운 상태를 기준으로 합니다. 명령은 스케줄러 컨테이너 안에서 airflow CLI 로 칩니다. 웹 화면은 http://localhost:8080(계정 airflow / airflow)입니다.

다 떴나

```bash
docker compose ps --format "table {{.Service}}\t{{.Status}}"
```

```text
SERVICE             STATUS
airflow-scheduler   Up About a minute (healthy)
airflow-triggerer   Up About a minute (healthy)
airflow-webserver   Up About a minute (health: starting)
airflow-worker      Up About a minute (healthy)
postgres            Up 2 minutes (healthy)
redis               Up 2 minutes (healthy)
```

웹서버는 다른 것보다 늦게 healthy 가 됩니다(실측). (health: starting) 이면 1~2분 더 기다립니다.

자주 쓰는 명령 (docker compose exec airflow-scheduler airflow …)

명령	하는 일	확인한 결과
version	버전	2.9.3
dags list	DAG 목록	내 DAG 와 함께 예제 DAG 수십 개가 보임
dags list-import-errors	읽지 못한 DAG 와 이유	파일 경로 + ModuleNotFoundError 등
tasks test hello_dag say_hello 2026-01-01	태스크 하나를 기록 없이 시험 실행	Marking task as SUCCESS
dags unpause hello_dag	예약 켜기	is_paused False
dags trigger hello_dag	지금 한 번 실행	run_type manual · state queued
dags list-runs -d hello_dag	실행 기록	state success (트리거 뒤 약 6초)
dags pause hello_dag	예약 끄기	is_paused True
users list	웹 계정	airflow · Admin

시험에 쓴 DAG

```python
# dags/hello_dag.py
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator

with DAG("hello_dag", start_date=datetime(2026, 1, 1), schedule=None, catchup=False) as dag:
    PythonOperator(task_id="say_hello", python_callable=lambda: print("hello from airflow"))
```

막힐 때 (실측)

- dags list 맨 위에 「Error: Failed to load all files」 — 읽지 못한 DAG 파일이 하나 이상 있다는 뜻입니다. dags list-import-errors 로 어느 파일이 왜 깨졌는지 봅니다. 목록 자체는 나머지 DAG 로 계속 나옵니다.
- 깨진 DAG 가 하나 있으면 tasks test 같은 다른 명령도 그 Traceback 을 먼저 길게 찍습니다. 결과 줄(Marking task as SUCCESS)은 그 아래에 있습니다.
- print("hello from airflow") 가 로그에 「hello from ***」 로 보입니다. 비밀번호(airflow)와 같은 글자를 Airflow 가 가려서 그렇습니다. 오류가 아닙니다(실측).
- dags list 에 예제 DAG 가 잔뜩 나오는 것은 공식 compose 가 AIRFLOW__CORE__LOAD_EXAMPLES: 'true' 이기 때문입니다. 내 DAG 만 보려면 dags list | grep 이름 으로 거릅니다.
- 새로 넣은 DAG 는 기본으로 멈춤(is_paused True) 상태입니다. trigger 해도 안 돌면 unpause 부터 합니다.
